Autonomous Systems: Xây dựng Autonomous Agent hoàn thành checklist ba bước

Trong làn sóng phát triển của Trí tuệ Nhân tạo hiện đại, khái niệm **AI Agent** đã có bước chuyển mình ngoạn mục từ những chuỗi câu lệnh tĩnh (Chains/Workflows) sang các hệ thống tự hành (**Autonomous Systems**). Một tác nhân tự hành hướng mục tiêu không chỉ thực thi một chuỗi hướng dẫn cố định được lập trình sẵn mà sở hữu khả năng tự phân tích mục tiêu (Goal-driven Planning), tự chia nhỏ nhiệm vụ thành các bước khả thi, tuần tự thực thi, truyền dữ liệu ngữ cảnh giữa các chặng và tự đánh giá để kích hoạt điều kiện dừng (**Stop Condition**) an toàn.

Bài học thực hành chuyên sâu này sẽ hướng dẫn bạn xây dựng trọn vẹn một **Autonomous Checklist Agent** với giới hạn hữu hạn (**Bounded Loop**) tối đa ba bước. Chúng ta sẽ lấy kịch bản thực tế rất trực quan và gần gũi: *Người dùng nhập yêu cầu viết một bài chia sẻ ngắn giải thích RAG (Retrieval-Augmented Generation) cho người mới bắt đầu.* Hệ thống tác nhân sẽ tự động lập checklist gồm 3 bước logic (Lên dàn ý → Viết nội dung → Rà soát & Hoàn thiện), lần lượt thực thi thông qua mô hình ngôn ngữ lớn (LLM cục bộ qua Ollama API, mặc định sử dụng `tinix-lm:latest`), ghi lại nhật ký (**nhật ký từng bước**) minh bạch và xuất ra báo cáo cuối cùng (**báo cáo tổng kết**).

Bài viết này dành cho ai?

- **Ai nên đọc bài này:** Kỹ sư AI (AI Engineers), lập trình viên Backend/Python, kỹ sư giải pháp tự động hóa muốn chuyển từ tư duy viết prompt thông thường sang xây dựng hệ thống tác nhân tự hành có khả năng tự lập kế hoạch và tự kiểm soát vòng lặp.
- **Yêu cầu kiến thức nền:** Hiểu biết cơ bản về ngôn ngữ lập trình Python (Data classes, Enum, xử lý chuỗi và tệp tin JSON). Đã từng làm quen với khái niệm LLM và kỹ thuật sinh văn bản có cấu trúc.
- **Yêu cầu hệ thống:** Máy tính cài đặt sẵn **Python 3.8+**. Để có thể tái tạo kết quả (reproducibility) trong các Integration Tests, yêu cầu cài đặt và khởi động Ollama cục bộ bằng lệnh `ollama serve` (cổng mặc định 11434, hoặc 11436). Do `tinix-lm:latest` là mô hình nội bộ độc quyền không được phát hành công khai (vui lòng liên hệ tác giả qua email admin@tinix.io để xin cấp quyền `Modelfile`), nếu bạn đã có `Modelfile`, hãy khởi tạo mô hình bằng lệnh `ollama create tinix-lm:latest`

`-f Modelfile`. Nếu không có quyền truy cập, bạn có thể chạy Agent bằng mô hình thay thế `qwen2.5:3b` (bằng lệnh `ollama pull qwen2.5:3b`). Nếu không có model nào hoặc Ollama không khả dụng, hệ thống sẽ tự động chuyển sang Deterministic Fallback. Hệ thống lưu vết chi tiết siêu dữ liệu (metadata) môi trường để đảm bảo độ tin cậy.

Mục tiêu:

- **Kiến thức đạt được:** Nắm vững kiến trúc cốt lõi của một hệ thống tác nhân tự hành (autonomous agent): bộ lập kế hoạch (planner), bộ thực thi tuần tự (executor), bộ nhớ trạng thái ngữ cảnh (context memory), bộ điều phối vòng lặp hữu hạn (bounded loop controller) và bộ đánh giá điều kiện dừng (stop condition evaluator). Phân biệt ranh giới rõ ràng giữa quy trình cứng (hardcoded workflow) và tác nhân tự thích ứng (autonomous agent).
- **Sản phẩm đầu ra:** Xây dựng thành công script `checklist_agent.py` hoàn chỉnh. Chạy thành công ca kiểm thử tạo bài viết chia sẻ về RAG với đầy đủ 3 thành phần đầu ra bắt buộc theo tiêu chuẩn: **kế hoạch ban đầu** (Initial Plan), **nhật ký từng bước** (Step Execution Logs) và **báo cáo tổng kết** (báo cáo tổng kết) kèm nội dung bài viết đã nghiệm thu xuất ra tệp tin `agent_final_report.json`.

Tác giả: Kỹ sư AI Phạm Thành Thái
AI Guru x TiniX

MỤC LỤC

Bài viết này dành cho ai? 1

Mục tiêu: 2

Các thuật ngữ cơ bản 4

Lý thuyết cơ bản: Kiến trúc tác nhân tự hành và vòng lặp quyết định 4

1. Phân biệt chuỗi tác vụ cố định (hardcoded chain) và tác nhân tự hành 4
2. Năm thành phần cốt lõi của một Checklist Agent 5

Thiết kế checklist tự động và cơ chế quản lý trạng thái 6

1. Kịch bản thực tế: Bài viết chia sẻ về RAG cho người mới bắt đầu 6
2. Cơ chế truyền trạng thái (context propagation) 7

Điều kiện dừng và giới hạn an toàn trong vòng lặp 7

1. Hiểm họa của vòng lặp vô hạn (infinite loop trap) 7
2. Thiết kế vòng lặp giới hạn và bộ đánh giá dừng đa trạng thái 8

Cài đặt mã nguồn chi tiết bằng Python 8

1. Định nghĩa các cấu trúc dữ liệu cốt lõi 9
2. Bộ kiểm định chất lượng đa tầng và kết nối mô hình ngôn ngữ lớn 9
3. Thiết kế khung ToolRegistry (Mock) làm điểm mở rộng 11
4. Lớp điều khiển tác nhân tự hành 11
5. Vòng lặp thực thi tự hành 13

Thực nghiệm kịch bản: Kiểm chứng năng lực tác nhân tự hành 14

1. Kịch bản 1: Viết bài chia sẻ hoàn chỉnh về RAG (3 bước đầy đủ) 14
2. Kịch bản 2: Mục tiêu nhỏ chuẩn bị outline Docker (tự xác định 2 bước và dừng sớm) 15
3. So sánh chất lượng sản phẩm giữa các bước 15
4. Phân rã kiểm thử: Offline Unit Tests và Integration Tests 16

Phân tích báo cáo tổng kết và nhật ký từng bước thực thi 17

Mở rộng và ứng dụng thực tế của tác nhân tự hành 18

Kết luận: Đánh giá phạm vi tự hành và hướng phát triển 18

Tài liệu tham khảo 19

Các thuật ngữ cơ bản

Bảng 1: Các thuật ngữ trong hệ thống tác nhân tự hành (autonomous systems)

Thuật ngữ	Định nghĩa trong ngữ cảnh Checklist Agent
Tác nhân tự hành (autonomous agent)	Thực thể AI có quyền tự chủ đưa ra quyết định: tự phân tích mục tiêu tổng thể, tự lập kế hoạch hành động, tự quan sát kết quả và điều chỉnh hành vi mà không cần con người can thiệp vào từng bước rẽ nhánh.
Lập kế hoạch theo mục tiêu (goal-driven planning)	Cơ chế lập kế hoạch hướng mục tiêu. Thay vì tuân theo các bước cố định, tác nhân tiếp nhận mục tiêu và xác định danh sách công việc (checklist) phù hợp trong giới hạn an toàn.
Vòng lặp giới hạn (bounded loop)	Vòng lặp thực thi có giới hạn chặn trên (ví dụ `max_steps = 3`). Đây là cơ chế chốt chặn an toàn bắt buộc trong hệ thống tự hành để triệt tiêu nguy cơ vòng lặp vô hạn (Infinite Loop) gây tốn kém chi phí token hoặc treo máy.
Điều kiện dừng (stop condition)	Bộ quy tắc logic xác định khi nào tác nhân phải ngưng vòng lặp. Trong triển khai này, các trạng thái dừng chính gồm: hoàn thành mục tiêu (`COMPLETED / Goal Achieved`), hết ngân sách bước (`BUDGET_EXHAUSTED`) và lỗi thực thi (`FAILED`).
Bộ nhớ trạng thái ngữ cảnh (context memory)	Cơ chế truyền và tích lũy trạng thái/dữ liệu giữa các bước thực thi. Kết quả đầu ra của Bước 1 (Dàn ý) trở thành đầu vào trực tiếp nuôi Bước 2 (Viết bài) và Bước 3 (Rà soát).
Nhật ký từng bước & báo cáo tổng kết	Bản ghi kiểm toán (Audit Trail) chi tiết từng chặng và báo cáo tổng kết cuối cùng, đảm bảo tính minh bạch, có thể quan sát (Observability) và kiểm chứng của hệ thống tự hành.

Lý thuyết cơ bản: Kiến trúc tác nhân tự hành và vòng lặp quyết định

1. Phân biệt chuỗi tác vụ cố định (hardcoded chain) và tác nhân tự hành

Trong giai đoạn đầu của kỹ thuật phần mềm AI, các lập trình viên thường xây dựng ứng dụng dưới dạng chuỗi pipeline tuần tự:

`Input → Prompt 1 → Model → Prompt 2 → Model → Output`

Mô hình này (thường thấy trong các sequential chain cơ bản) hoạt động ổn định với các tác vụ đơn biến, nhưng bộc lộ điểm yếu chết người: **hoàn toàn không có tính thích ứng**. Nếu gặp một mục tiêu yêu cầu chỉ 2 bước xử lý, hệ thống vẫn ép buộc chạy qua bước thứ 3; ngược lại, nếu một bước bị lỗi hoặc thiếu dữ liệu, toàn bộ chuỗi sẽ sụp đổ như quân cờ domino.



Hình 1: Kiến trúc tổng thể của tác nhân tự hành với vòng lặp giới hạn (bounded loop)

Ngược lại, **tác nhân tự hành (autonomous agent)** hoạt động dựa trên vòng lặp phản hồi khép kín (Sense - Plan - Act - Observe - Evaluate). Điểm mấu chốt của tính tự hành là khả năng lựa chọn hành động dựa trên mục tiêu và trạng thái hiện tại, thực thi, quan sát kết quả, đánh giá và điều chỉnh hoặc lập lại kế hoạch trong giới hạn an toàn.

2. Năm thành phần cốt lõi của một Checklist Agent

Một hệ thống tác nhân tự hành hoàn chỉnh được cấu thành từ 5 bộ phận hữu cơ:

- **Tiếp nhận mục tiêu (goal ingestion):** Nhận câu lệnh hoặc bài toán từ người dùng (ví dụ: *"Viết một bài chia sẻ ngắn giải thích RAG là gì cho người mới bắt đầu"*).
- **Bộ lập kế hoạch động (dynamic planner):** Sử dụng mô hình ngôn ngữ lớn phân tích ý định và sản sinh danh sách công việc (checklist) với ràng buộc nghiêm ngặt: độ dài

checklist không vượt quá `max_steps` (≤ 3 bước).

- **Bộ thực thi tuần tự (sequential executor):** Chịu trách nhiệm thực thi từng bước trong checklist theo chỉ dẫn hành động (action) dựa trên ngữ cảnh tích lũy từ các bước trước.
- **Bộ nhớ trạng thái ngữ cảnh (context memory):** Đóng vai trò cầu nối dữ liệu, tích lũy kết quả của các bước trước đó để làm đầu vào cho các bước tiếp theo.
- **Bộ đánh giá dừng và báo cáo (stop evaluator & reporter):** Giám sát vòng lặp, kiểm soát điều kiện dừng an toàn và tổng hợp toàn bộ tiến trình thành báo cáo nghiệm thu cuối cùng.

Thiết kế checklist tự động và cơ chế quản lý trạng thái

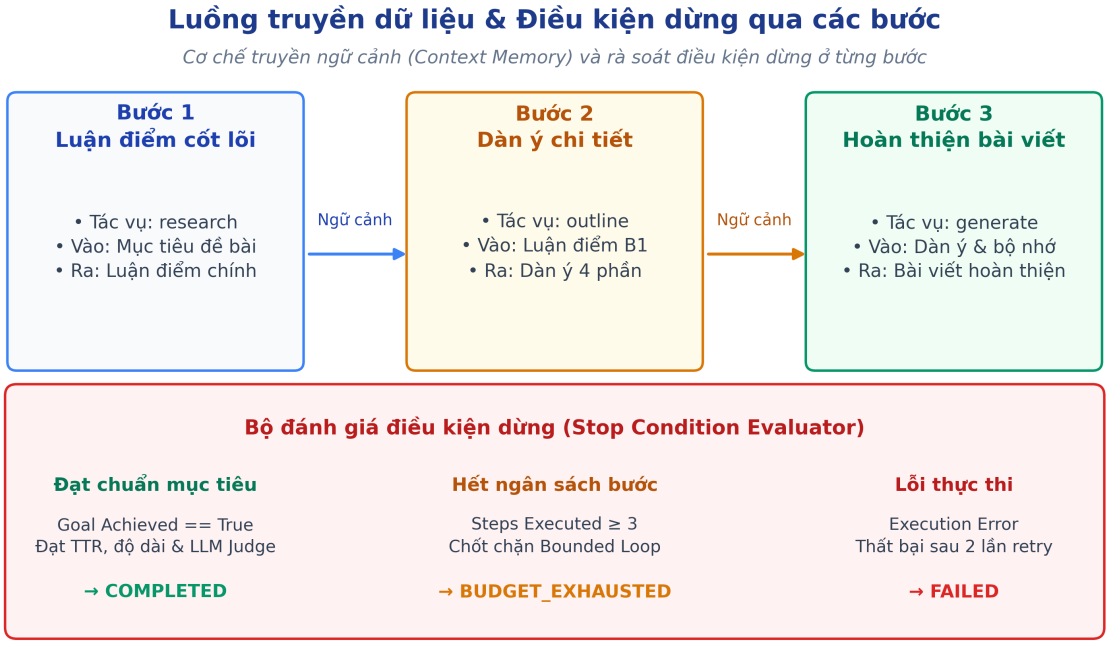
1. Kịch bản thực tế: Bài viết chia sẻ về RAG cho người mới bắt đầu

Để chứng minh năng lực tự hành một cách thuyết phục nhất, chúng ta chọn một tác vụ cụ thể, có ranh giới rõ ràng nhưng đòi hỏi chất lượng logic cao: **"Viết một bài chia sẻ ngắn giải thích RAG là gì cho người mới bắt đầu"**.

Hệ thống thiết kế hai phương án phân rã checklist 3 bước tương ứng: (1) **Kịch bản lập kế hoạch động qua LLM (Dynamic Planning)** trong phiên chạy thực tế (`research_points` → `outline` → `generate`, phân tích chi tiết tại mục "Thực nghiệm kịch bản" ở trang 14); và (2) **Kịch bản dự phòng cấu trúc xác định (Deterministic Fallback Plan)** làm chốt chặn an toàn khi LLM ngoại tuyến hoặc phản hồi sai định dạng (`outline` → `generate` → `review_polish`). Dưới đây là phân rã của kịch bản fallback cấu trúc:

- **Bước 1: Xác định nội dung cơ bản của RAG (Action: `outline`).** Thu thập và làm rõ các luận điểm cốt lõi: khái niệm RAG, vì sao cần RAG và cơ chế 3 chặng (Indexing → Retrieval → Generation).
- **Bước 2: Xây dựng dàn ý bài chia sẻ ngắn (Action: `generate`).** Kế thừa luận điểm từ Bước 1 để cấu trúc hóa dàn bài 4 phần: Mở đầu, Bản chất kỹ thuật, Ví dụ minh họa và Lời kết thực hành.
- **Bước 3: Viết và chỉnh sửa hoàn thiện bài chia sẻ (Action: `review_polish`).** Kế thừa dàn ý từ Bước 2, chấp bút hoàn chỉnh bài viết, lồng ghép ví dụ trực quan (thi "mở sách" vs "đóng sách") và kiểm định qua `QualityEvaluator`.

Khác biệt cốt lõi so với workflow cố định: Ở chế độ LLM, Planner phân rã động từ mục tiêu (LLM tự chủ chọn `generate` làm bước hoàn thiện); cơ chế fallback định sẵn chỉ đóng vai trò chốt chặn an toàn khi LLM ngoại tuyến. Cả hai đều hội tụ về việc nghiệm thu sản phẩm qua `QualityEvaluator`.



Hình 2: Luồng truyền dữ liệu qua bộ nhớ trạng thái và cơ chế rà soát điều kiện dừng ở từng bước (Hành động Bước 3 linh hoạt: `generate` trong kịch bản LLM thực tế hoặc `review_polish` trong kịch bản dự phòng)

2. Cơ chế truyền trạng thái (context propagation)

Như minh họa ở Hình 2, tính liên tục của tác nhân phụ thuộc hoàn toàn vào việc chia sẻ bộ nhớ `context_memory`:

- Khi Bước 1 hoàn thành, các luận điểm nền tảng được lưu vào `research_points` (hoặc `outline`) trong `context_memory`.
- Bước 2 khi bắt đầu sẽ tự động trích xuất ngữ cảnh Bước 1 để định hướng sinh dàn ý phân cấp (`outline`) hoặc bản thảo ban đầu (`generate`).
- Bước 3 kết hợp ngữ cảnh tích lũy từ các bước trước, tiến hành hoàn thiện toàn diện bài viết (`generate` hoặc `review_polish`), kiểm định theo bộ tiêu chí `QualityEvaluator` và xuất ra kết quả nghiệm thu tại `final_article`.

Nếu cơ chế truyền trạng thái này bị đứt gãy, các bước sau sẽ hoạt động thiếu định hướng và tác nhân sẽ không thể đạt được mục tiêu chung.

Điều kiện dừng và giới hạn an toàn trong vòng lặp

1. Hiểm họa của vòng lặp vô hạn (infinite loop trap)

Trong các hệ thống tự hành không có kiểm soát, mô hình ngôn ngữ rất dễ rơi vào trạng thái "ảo giác lặp" (reasoning loop): bước sau cố gắng sửa đổi bước trước một cách không hồi

kết, hoặc liên tục gọi lại cùng một công cụ với tham số tương tự. Điều này dẫn tới hai hậu quả nghiêm trọng:

1. **Cạn kiệt ngân sách (Token exhaustion):** Tiêu tốn chi phí gọi mô hình lớn một cách không cần thiết.
2. **Tắc nghẽn hệ thống (Process hanging):** Tài nguyên server bị chiếm dụng, luồng xử lý không bao giờ trả về kết quả cho người dùng.

2. Thiết kế vòng lặp giới hạn và bộ đánh giá dừng đa trạng thái

Để giải quyết triệt để rủi ro trên, hệ thống của chúng ta thiết lập cơ chế kiểm soát kép phân định rõ ràng 3 trạng thái kết thúc:

1. **Trạng thái hoàn thành mục tiêu (`COMPLETED` / `GOAL_ACHIEVED`):** Sau mỗi bước, hàm `_evaluate_stop_condition()` đánh giá xem sản phẩm tạo ra có thỏa mãn điều kiện nghiệm thu theo đúng mục tiêu hay không: ở chế độ LLM (`use_llm=True`), hệ thống siết chặt điều kiện nghiệm thu bắt buộc phải đạt xác minh ngữ nghĩa (`semantic_verified=True`); ở chế độ offline (`use_llm=False`), hệ thống nghiệm thu theo chuẩn logic định lượng và ghi rõ chưa qua thẩm định ngữ nghĩa. Khi thỏa mãn, hệ thống kích hoạt dừng với lý do `Goal achieved` (trạng thái `COMPLETED`). Đồng thời, trong khâu tổng hợp báo cáo cuối cùng, hệ thống kiểm tra tính nhất quán lần nữa: nếu sản phẩm chưa đạt chuẩn chất lượng hoặc thiếu xác minh ngữ nghĩa, trạng thái được cập nhật hạ cấp về `BUDGET_EXHAUSTED` cùng lý do dừng chi tiết, tuyệt đối không để lọt trạng thái `COMPLETED` khi kết quả kiểm định không đạt.
2. **Trạng thái hết ngân sách bước (`BUDGET_EXHAUSTED`):** Khi số bước thực thi đạt ngưỡng cho phép (ví dụ `max_steps = 1` với mục tiêu viết bài, hệ thống mới chỉ xong dàn ý), tác nhân ngắt vòng lặp an toàn và ghi nhận trạng thái `BUDGET_EXHAUSTED` chứ không báo nhầm là `COMPLETED`.
3. **Trạng thái thất bại (`FAILED`):** Khi một bước thực thi gặp sự cố ngoại lệ (Action failure), Agent tự động thử lại một lần. Nếu lỗi liên tiếp vượt ngưỡng cho phép, hệ thống mới chuyển trạng thái sang `FAILED` và lưu vết lỗi vào nhật ký từng bước (Step Log).

Nhờ đó, tác nhân vừa có tính linh hoạt tự hành, vừa tăng khả năng kiểm soát trong phạm vi các giới hạn được cấu hình.

Cài đặt mã nguồn chi tiết bằng Python

Toàn bộ hệ thống được đóng gói gọn gàng trong tệp `checklist_agent.py` với cấu trúc hướng đối tượng rõ ràng, sử dụng triệt để thư viện tiêu chuẩn của Python để đảm bảo tính

gọn nhẹ và tương thích cao nhất.

1. Định nghĩa các cấu trúc dữ liệu cốt lõi

Chúng ta sử dụng `dataclass` và `Enum` để định nghĩa kiểu dữ liệu chặt chẽ cho trạng thái, từng bước trong kế hoạch, nhật ký thực thi và báo cáo cuối cùng:

```
class AgentStatus(str, Enum):
    IDLE = "IDLE"
    PLANNING = "PLANNING"
    RUNNING = "RUNNING"
    COMPLETED = "COMPLETED"
    BUDGET_EXHAUSTED = "BUDGET_EXHAUSTED"
    FAILED = "FAILED"

@dataclass
class Step:
    step_id: int; title: str; description: str; action_type: str
    status: str = "PENDING"; result: Optional[str] = None

@dataclass
class StepLog:
    step_id: int; step_title: str; timestamp: str; duration_sec: float
    input_context_keys: List[str]; output_summary: str; status: str
    details: Dict[str, Any] = field(default_factory=dict)

@dataclass
class FinalReport:
    goal: str; total_steps_planned: int; steps_executed: int
    status: AgentStatus; stop_reason: str
    plan: List[Dict[str, Any]]; execution_logs: List[Dict[str, Any]]
    final_output: str; quality_metrics: Dict[str, Any]; total_duration_sec: float
```

2. Bộ kiểm định chất lượng đa tầng và kết nối mô hình ngôn ngữ lớn

Thay vì sử dụng các chuỗi hardcode, tác nhân tích hợp `QualityEvaluator` tính toán trực tiếp các chỉ số kỹ thuật và kết nối mô hình ngôn ngữ lớn qua `OllamaClient`. **(Đoạn mã rút gọn minh họa logic kiểm định cốt lõi; triển khai đầy đủ trên GitHub trả về cả 9 trường: passed, quality_passed, semantic_verified, char_count, word_count, ttr, matched_keywords, judge_verdict, review_notes)**:

```
class QualityEvaluator:
    @staticmethod
    def calculate_ttr(text: str) -> float:
        # Đo Type-Token Ratio (TTR) theo chuẩn tách từ lọc ký tự đặc biệt
        words = [re.sub(r"^[^\w]+|[^\w]+$", "", w) for w in text.lower().split()
                 if len(re.sub(r"^[^\w]+|[^\w]+$", "", w)) > 0]
        return round(len(set(words)) / len(words), 3) if words else 0.0

    @classmethod
    def evaluate(cls, content: str, goal: str, intent: str, llm_client=None, source_doc:
    ↪ Optional[str]=None, require_llm_judge: bool=False) -> Dict[str, Any]:
        if not content: return {"passed": False, "ttr": 0.0, "char_count": 0}
        words = [re.sub(r"^[^\w]+|[^\w]+$", "", w) for w in content.lower().split() if w]
```

```
        ttr = cls.calculate_ttr(content)
        min_chars = 100 if intent in ("outline_only", "summarize") else 250
        crit_length = len(content) >= min_chars
        crit_structure = bool(re.search(r'(^|\n)(#{1,4}\s+|[0-9]+\.\s+|-\s+|\*\s+)', content))
        crit_diversity = ttr >= 0.35
        stop_words = {"viết", "bài", "cho", "về", "người", "mới", "bắt", "đầu", "sinh",
                      "viên", "và", "là", "gì", "một", "tài", "liệu", "chuẩn", "bị", "hướng", "dẫn",
                      "này", "đây"}
        goal_kws = [w for w in goal.lower().split() if len(w) > 2 and w not in stop_words]
        matched_kws = [kw for kw in goal_kws if kw in content.lower()]
        crit_relevance = len(matched_kws) > 0 if goal_kws else True
        # 5. Kiểm tra lỗi cắt xén câu (cut-off sentence)
        crit_complete = not (content.strip().endswith((",", "và", "hoặc", "là")) or "..." in
    content.strip()[-5:])
        quality_passed = crit_length and crit_structure and crit_diversity and crit_relevance and
    crit_complete
        # 6. Đánh giá bằng LLM-as-a-judge (Cơ chế fail-closed & tách bạch cờ)
        semantic_verified = False; judge_verdict = None
        has_source = bool(source_doc and source_doc.strip())
        judge_needed = require_llm_judge or (llm_client is not None)
        if judge_needed:
            if llm_client is None or not llm_client.is_available():
                judge_verdict = {"judge_status": "LLM_UNAVAILABLE", "correctness": 0, "groundedness": 0,
    "completeness": 0,
                                 "judge_model": getattr(llm_client, "model", "none") if llm_client else
    "none",
                                 "groundedness_source": "external_document" if has_source else
    "none_open_domain",
                                 "reason": "Dịch vụ LLM không khả dụng (Fail-closed: từ chối nghiệm thu
    khi thiếu Judge)"}
                return {"passed": False, "quality_passed": quality_passed, "semantic_verified": False,
    "judge_verdict": judge_verdict, ...} # đủ 9 trường
            llm_eval = llm_client.generate(prompt)
            if llm_eval and re.search(r'{.*}', llm_eval, re.DOTALL):
                try:
                    judge_res = json.loads(re.search(r'{.*}', llm_eval, re.DOTALL).group(0))
                    req_keys = ("correctness", "groundedness", "completeness")
                    types_ok = all(k in judge_res and type(judge_res[k]) is int and judge_res[k] in (0,
    1) for k in req_keys)
                    all_ones = types_ok and all(judge_res[k] == 1 for k in req_keys)
                    judge_verdict = judge_res
                    judge_verdict["judge_model"] = getattr(llm_client, "model", "tinix-lm:latest")
                    judge_verdict["groundedness_source"] = "external_document" if has_source else
    "none_open_domain"
                    if all_ones:
                        semantic_verified = True; judge_verdict["judge_status"] = "VERIFIED_SUCCESS"
                    else:
                        semantic_verified = False; judge_verdict["judge_status"] = "REJECTED"
                except Exception as e:
                    semantic_verified = False; judge_verdict = {"judge_status": "PARSE_EXCEPTION",
    "error": str(e)}
            else:
                semantic_verified = False; judge_verdict = {"judge_status": "PARSE_ERROR", "error":
    "Parse fail"}
            passed = quality_passed and semantic_verified
        else:
            heuristic_passed = not any(k in content.lower() for k in ["chính xác 100%", "tuyệt đối"])
```

```
            judge_verdict = {"judge_status": "OFFLINE_HEURISTIC_PASS" if heuristic_passed else
↪  "REJECTED_OFFLINE",
                            "correctness": 1 if heuristic_passed else 0, "groundedness": 1 if has_source
↪  else None,
                            "completeness": 1, "judge_model": "none (offline_heuristic)",
                            "groundedness_source": "external_document" if has_source else
↪  "none_open_domain"}
            passed = quality_passed and heuristic_passed
        return {"passed": passed, "quality_passed": quality_passed, "semantic_verified":
↪  semantic_verified, "judge_verdict": judge_verdict, ...} # đủ 9 trường
```

Nguyên tắc phân định và Fail-Closed: Hệ thống tách bạch rõ ràng giữa hai chỉ số: `quality_passed` (kiểm định định lượng về độ dài, cấu trúc, TTR) và `semantic_verified` (xác thực ngữ nghĩa bởi LLM Judge). Khi `use_llm=True` mà LLM ngoại tuyến hoặc lỗi, cơ chế Fail-Closed kích hoạt: `semantic_verified=False` và từ chối nghiệm thu (`passed=False`). Với tác vụ tri thức mở (`input_document=None`), tiêu chí Groundedness được LLM Judge thẩm định theo tri thức nội tại (`none_open_domain`), tuyệt đối không ngộ nhận là đã đối chiếu tài liệu nguồn ngoài hệ thống (triển khai trên GitHub kết xuất trọn vẹn 9 trường của `EvaluationResult`).

3. Thiết kế khung ToolRegistry (Mock) làm điểm mở rộng

`ToolRegistry` được thiết kế như khung thử nghiệm (sandbox) phục vụ mở rộng công cụ và cô lập vòng lặp tự hành, bảo đảm chốt chặn bounded loop mà không phụ thuộc mạng ngoài. Các công cụ tìm kiếm và đọc tài liệu hoạt động dạng mock; công cụ `calculate` dùng bộ phân tích cú pháp `ast` an toàn thay cho `eval()`, loại bỏ hoàn toàn nguy cơ thực thi mã độc:

```python
class ToolRegistry:
    @staticmethod
    def search_knowledge(topic: str) -> str:
        return f"[Mock Tool: search] Tìm thấy kết quả: {topic}"

    @staticmethod
    def calculate(expr: str) -> str:
        try:
            def _safe_eval(node):
                if isinstance(node, ast.Num): return node.n
                if isinstance(node, ast.BinOp) and isinstance(node.op, ast.Add):
                    return _safe_eval(node.left) + _safe_eval(node.right)
                raise ValueError("Toán tử không hợp lệ")
            return f"[Mock Tool: calc] Kết quả: {_safe_eval(ast.parse(expr, mode='eval').body)}"
        except Exception:
            return "[Mock Tool: calc] Lỗi tính toán biểu thức."
```

4. Lớp điều khiển tác nhân tự hành

Lớp `AutonomousChecklistAgent` kết hợp LLM để tự động phân rã kế hoạch theo mục tiêu và đánh giá nghiệm thu đa trạng thái:

```python
class AutonomousChecklistAgent:
    def __init__(self, max_steps: int = 3, use_llm: bool = True):
        if max_steps > 3: raise ValueError("Vi phạm Bounded Loop: max_steps > 3!")
        self.max_steps, self.use_llm = min(max(1, max_steps), 3), use_llm
        self.llm = OllamaClient() if use_llm else None; self.evaluator = QualityEvaluator()
        self.context_memory: Dict[str, Any] = {}

    def plan_steps(self, goal: str, max_steps: int = None) -> List[Step]:
        self.status = AgentStatus.PLANNING
        topic, intent = self._extract_topic_and_intent(goal)
        steps: List[Step] = []
        # 1. Gọi trực tiếp LLM Ollama sinh checklist JSON động
        if self.use_llm and self.llm and self.llm.is_available():
            prompt = f"Lập checklist tối đa 3 bước cho mục tiêu: '{goal}'. Trả về JSON..."
            resp = self.llm.generate(prompt, max_tokens=320)
            if resp:
                try:
                    m = re.search(r"\[\s*{.*}\s*\]", resp, re.DOTALL)
                    if m:
                        raw = json.loads(m.group(0))[:(max_steps or self.max_steps)]
                        steps = [Step(i, s.get("title", f"Bước {i}"), s.get("description", ""),
↪  s.get("action_type", "generate")) for i, s in enumerate(raw, 1)]
                except Exception: steps = []
        # 2. Fallback cấu trúc xác định khi LLM ngoại tuyến
        if not steps:
            if intent == "outline_only": steps = [Step(1, f"Luận điểm: {topic}", "", "research_points"),
             ↪  Step(2, f"Dàn ý: {topic}", "", "outline")]
            elif intent == "summarize": steps = [Step(1, f"Ý chính: {topic}", "", "research_points"),
             ↪  Step(2, f"Tóm tắt: {topic}", "", "summarize")]
            else: steps = [Step(1, f"Dàn ý {topic}", "", "outline"), Step(2, f"Soạn {topic}", "",
             ↪  "generate"), Step(3, f"Rà soát {topic}", "", "review_polish")]
        self.plan = steps[:(max_steps or self.max_steps)]
        return self.plan

    def _evaluate_stop_condition(self, step_idx: int, total_steps: int, intent: str, goal: str) ->
     ↪  Tuple[bool, str, Optional[AgentStatus]]:
        src = self.context_memory.get("input_document")
        # 1. Nghiệm thu sớm: Chỉ khi QualityEvaluator.passed == True
        for k, name in [("outline", "Dàn ý"), ("final_article", "Bài viết")]:
            if (intent == "outline_only" if k == "outline" else True) and k in self.context_memory:
                if self.evaluator.evaluate(self.context_memory[k], goal, intent, self.llm,
                 ↪  source_doc=src, require_llm_judge=self.use_llm)["passed"]:
                    return True, f"Goal achieved: {name} đạt chuẩn định lượng.", AgentStatus.COMPLETED
        # 2. Hết ngân sách bước thực thi: Bắt buộc thẩm định lại chất lượng
        if step_idx >= total_steps:
            target = self.context_memory.get("final_article") or self.context_memory.get("outline")
            if target and self.evaluator.evaluate(target, goal, intent, self.llm, source_doc=src,
             ↪  require_llm_judge=self.use_llm)["passed"]:
                return True, "Goal achieved: Sản phẩm đạt chuẩn nghiệm thu.", AgentStatus.COMPLETED
            return True, "Budget exhausted: Hết bước nhưng chưa đạt chuẩn nghiệm thu.",
             ↪  AgentStatus.BUDGET_EXHAUSTED
        return False, "Continue", None
```

5. Vòng lặp thực thi tự hành

Phương thức `run()` quản trị trọn vẹn chu trình làm việc tự động khép kín kèm cơ chế bắt lỗi an toàn:

```
    def run(self, goal: str, input_document: Optional[str] = None) -> FinalReport:
        start_time = time.perf_counter()
        if input_document: self.context_memory["input_document"] = input_document.strip()
        plan, (topic, intent) = self.plan_steps(goal), self._extract_topic_and_intent(goal)
        all_planned_steps, action_queue, stop_reason = plan.copy(), plan.copy(), ""
        self.status, consecutive_failures, steps_executed = AgentStatus.RUNNING, 0, 0
        while action_queue and steps_executed < self.max_steps:
            step = action_queue.pop(0); steps_executed += 1
            try:
                result = self._execute_step_action(step, topic, goal); step.status = "COMPLETED"
                consecutive_failures = 0
            except Exception:
                consecutive_failures += 1
                if consecutive_failures >= 2:
                    step.status, self.status = "FAILED", AgentStatus.FAILED
                    stop_reason = "Lỗi liên tiếp"; break
                action_queue.insert(0, step); continue
            if action_queue and self._observe_and_adapt(step, result, action_queue[0], goal) == "REPLAN":
                rem = self.max_steps - steps_executed
                if rem > 0:
                    action_queue, self.status = self.plan_steps(f"Khắc phục: {goal}", max_steps=rem),
↪  AgentStatus.RUNNING
                    all_planned_steps.extend(action_queue); continue
                else:
                    self.status, stop_reason = AgentStatus.BUDGET_EXHAUSTED, f"Budget exhausted: Hết bước khi REPLAN
                     ↪  ({steps_executed}/{self.max_steps})."; break
            should_stop, reason, final_status = self._evaluate_stop_condition(steps_executed, len(all_planned_steps),
↪  intent, goal)
            if should_stop:
                stop_reason, self.status = reason, final_status or self.status; break
        if self.final_output:
            self.quality_metrics = self.evaluator.evaluate(self.final_output, goal, intent, self.llm,
             ↪  source_doc=self.context_memory.get("input_document"), require_llm_judge=self.use_llm)
        else:
            self.quality_metrics = {"passed": False, "review_notes": ["Không có sản phẩm đầu ra."]}

        # Đảm bảo nhất quán trạng thái và cập nhật lý do dừng chi tiết
        final_passed = self.quality_metrics.get("passed", False)
        semantic_ok = self.quality_metrics.get("semantic_verified", False) if self.use_llm else True
        failed_notes = [n for n in self.quality_metrics.get("review_notes", []) if n.startswith("✗")]
        reason_detail = "; ".join(failed_notes) if failed_notes else ("Chưa đạt xác minh LLM Judge" if not semantic_ok
↪  else "Chưa đạt chất lượng")

        if self.status in (AgentStatus.RUNNING, AgentStatus.PLANNING):
            if final_passed and semantic_ok:
                self.status = AgentStatus.COMPLETED
                if not stop_reason: stop_reason = f"Goal achieved: Đạt chuẩn sau {steps_executed} bước."
            else:
                self.status = AgentStatus.BUDGET_EXHAUSTED
                if not stop_reason: stop_reason = f"Budget exhausted: Hết bước ({reason_detail})."

        # Kiểm tra tính nhất quán cuối cùng: Hạ COMPLETED nếu kiểm tra lại báo cáo thất bại
        if self.status == AgentStatus.COMPLETED and (not final_passed or not semantic_ok):
            self.status, stop_reason = AgentStatus.BUDGET_EXHAUSTED, f"Budget exhausted: Chưa đạt nghiệm thu sau kiểm
             ↪  tra lại ({reason_detail})."

        return FinalReport(goal=goal, total_steps_planned=len(all_planned_steps), steps_executed=steps_executed,
         ↪  status=self.status, stop_reason=stop_reason, plan=[asdict(s) for s in all_planned_steps],
         ↪  execution_logs=[asdict(l) for l in self.logs], final_output=self.final_output,
         ↪  quality_metrics=self.quality_metrics, total_duration_sec=round(time.perf_counter()-start_time, 3))
```

Thực nghiệm kịch bản: Kiểm chứng năng lực tác nhân tự hành

1. Kịch bản 1: Viết bài chia sẻ hoàn chỉnh về RAG (3 bước đầy đủ)

Người dùng nhập mục tiêu: *"Viết một bài chia sẻ ngắn giải thích RAG là gì cho người mới bắt đầu"*. Planner tự động phân rã checklist 3 bước (`research_points` → `outline` → `generate`), thực thi và thích ứng như minh họa ở Hình 3:

```
thaipt@tinix-aiguru: ~/checklist-agent (python3 checklist_agent.py)
$ python3 code/checklist_agent.py

[AGENT] BẮT ĐẦU NHIỆM VỤ | tinix-lm:latest (Ollama)
MỤC TIÊU: Viết bài chia sẻ ngắn giải thích RAG cho người mới.
GIỚI HẠN: Tối đa 3 bước thực thi (Bounded Execution Loop)

[PLANNER] Khởi tạo checklist 3 bước động bằng LLM:
  [1] (research_points) Luận điểm cốt lõi về RAG
  [2] (outline) Dàn ý 4 phần (mở, thân, ví dụ, kết)
  [3] (generate) Hoàn thiện bài viết theo dàn ý

[BƯỚC 1] research_points -> [OK] COMPLETED (3.176s)
[BƯỚC 2] outline         -> [OK] COMPLETED (3.356s)
[BƯỚC 3] generate        -> [OK] COMPLETED (7.380s | 1.872 ký tự)
  [LLM JUDGE] Thẩm định: correctness=1, quality=PASS
[STOP CONDITION] Kích hoạt dừng: Goal achieved (Đạt chuẩn)

BÁO CÁO: COMPLETED | 3/3 bước | 19.812s | TTR=0.43 | PASS
```

Hình 3: Giao diện phiên thực thi trên terminal đối chiếu đồng bộ với dữ liệu kiểm toán thực tế (`test_execution_ollama.log` và `agent_final_report.json`)

Nhật ký thực thi minh họa 3 chặng dưới sự điều khiển của LLM `tinix-lm:latest` cục bộ:

1. **Lập kế hoạch động qua LLM (dynamic planning):** Agent gửi mục tiêu đến LLM để phân rã checklist 3 bước JSON: Bước 1 (`research_points`), Bước 2 (`outline`), Bước 3 (`generate`). Khác với fallback định sẵn, LLM linh hoạt chọn `generate` làm bước hoàn thiện bài viết để nghiệm thu.
2. **Quan sát và thích ứng bước kế tiếp (Observe & Adapt):** Sau Bước 1, Agent ghi nhận kết quả và chèn thích ứng vào prompt Bước 2: *"Trình bày RAG đơn giản, dễ hiểu bằng ví dụ đời sống"*. Sau Bước 2, Agent tiếp tục định hướng Bước 3: *"Nêu rõ RAG sử dụng thông tin thực tế từ nguồn bên ngoài"*. Nếu phát hiện bất thường, nhánh **REPLAN** sẽ lập lại kế hoạch, khôi phục `RUNNING` và cập nhật `action_queue`. Biến `steps_executed` đếm số bước thực chạy (≤ 3); `total_steps_planned` lưu tổng bước qua các lần lập (gồm cả REPLAN). Báo cáo kiểm toán hiển thị minh bạch dạng `steps_executed/total_steps_planned (Giới hạn: max_steps)`.
3. **Thực thi LLM thật và kiểm định dừng (real execution & quality gate):** Bước 3 chạy hoàn tất (thời gian benchmark 19.8s). Bộ `QualityEvaluator` thẩm định: TTR = 0.43 (≥ 0.35), 1.872 ký tự (391 từ), Markdown chuẩn và kích hoạt dừng `Goal achieved`.

2. Kịch bản 2: Mục tiêu nhỏ chuẩn bị outline Docker (tự xác định 2 bước và dừng sớm)

Để minh chứng Agent **không bắt buộc chạy đủ 3 bước trong mọi mục tiêu**, ta kiểm thử mục tiêu nhỏ: *"Chuẩn bị outline cho bài viết về Docker cho sinh viên IT."* Khi chạy ở chế độ dự phòng (`use_llm=False`):

- **Phân rã kế hoạch 2 bước khi chạy fallback:** Agent áp dụng luật nhận diện ý định (`outline_only`) để xác định mục tiêu chỉ cần đúng **2 bước**: Bước 1 (`research_points`: xác định luận điểm cốt lõi) → Bước 2 (`outline`: chuẩn hóa dàn ý chi tiết).
- **Kích hoạt điều kiện dừng sớm (stop condition):** Ngay khi Bước 2 hoàn thành, `QualityEvaluator` trực tiếp thẩm định dàn ý (độ dài, từ khóa Docker, cấu trúc phân cấp). Khi đạt chuẩn định lượng (`passed == True`), hệ thống lập tức kích hoạt điều kiện dừng: *"Goal achieved: Dàn ý hoàn chỉnh đạt chuẩn định lượng của QualityEvaluator."* Agent kết thúc thành công ở Bước 2, loại bỏ hoàn toàn các bước thừa và dừng an toàn.

3. So sánh chất lượng sản phẩm giữa các bước

Bảng 2 đối chiếu sự tiến hóa rõ rệt về chất lượng nội dung giữa dàn ý sau Bước 2 và bài viết hoàn thiện sau Bước 3 nhờ cơ chế truyền ngữ cảnh và vòng lặp quan sát thích ứng:

Bảng 2: Đối chiếu chất lượng nội dung qua chuỗi truyền trạng thái

Sản phẩm sau Bước 2 (Dàn ý chi tiết)	Sản phẩm sau Bước 3 (Bài viết hoàn thiện)
Khung cấu trúc 4 phần: mở đầu, giải thích RAG, ví dụ thực tế và lời kết.	Triển khai thành bài viết hoàn chỉnh với văn phong tự nhiên, mạch lạc cho người mới.
Định hình cấu trúc và ví dụ minh họa dạng gạch đầu dòng.	Diễn giải sâu sắc bằng ví dụ so sánh đời thường: học sinh thi "mở sách" vs "đóng sách" giúp người mới dễ hiểu ngay.
Chưa phải nội dung văn bản hoàn chỉnh (dừng ở mức khung dàn bài).	Hoàn thiện toàn bộ văn bản đạt 1.872 ký tự (391 từ), cấu trúc Markdown phân cấp rõ ràng.
Chưa qua khâu nghiệm thu chất lượng.	Đạt chuẩn định lượng bằng `QualityEvaluator`: TTR = 0.43 (đáp ứng ngưỡng đa dạng từ vựng ≥ 0.35), đủ từ khóa chủ đề và LLM Judge xác nhận.

4. Phân rã kiểm thử: Offline Unit Tests và Integration Tests

Nhằm đáp ứng tiêu chuẩn phần mềm, bộ kiểm thử `eval_agent.py` được phân tách rõ ràng thành hai nhóm: **Unit Tests (chạy offline)** không phụ thuộc mô hình để thẩm định cơ chế tự hành, vòng lặp an toàn và khả năng phục hồi lỗi; **Integration Tests** yêu cầu kết nối Ollama thực tế để đánh giá năng lực tác nhân. Toàn bộ kịch bản được chuẩn hóa tự động qua `unittest` và có điều kiện bỏ qua tự động (skip conditions) linh hoạt theo cấu hình môi trường:

Bảng 3: Kết quả kiểm thử 18 kịch bản (13 Offline Unit Tests đạt & 5 Real LLM Integration Tests)

STT	Kịch bản kiểm thử (Test Scenario)	Trạng thái	Điều kiện dừng / Kết quả
Nhóm 1: Offline Unit Tests & Plumbing Tests (13 kịch bản – Luôn Đạt 13/13 PASS)			
1	Chuẩn bị outline Docker (2 bước)	`PASS (COMPLETED)`	`Goal achieved` (Dừng sớm an toàn)
2	Tóm tắt tài liệu nguồn MCP (2 bước)	`PASS (COMPLETED)`	`Goal achieved` (Dùng `input_doc`)
3	Hết ngân sách bước (`max_steps=1`)	`PASS (EXHAUSTED)`	`Budget exhausted` (Không báo nhầm)
4	Kiểm duyệt từ chối (Chất lượng kém)	`PASS (EXHAUSTED)`	`Budget exhausted` (Chặn bài kém chuẩn)
5	Mục tiêu ngoài chủ đề mẫu (Kafka)	`PASS (COMPLETED)`	`Goal achieved` (Trích xuất chủ đề mở)
6	Quan sát & Thích ứng bước kế tiếp	`PASS (Observe)`	`Step Adapt` (Bước 1 chỉnh Bước 2)
7	Sự cố thực thi và phát hiện kẹt	`PASS (FAILED)`	`Execution failed` (Retry & dừng an toàn)
8	Rào chắn Bounded Loop & AST an toàn	`PASS (Guard)`	Chặn đứng vi phạm (`ValueError`, Pow limit)
9	Từ chối tóm tắt khi thiếu tài liệu nguồn	`PASS (FAILED)`	`Execution failed` (Chặn nội dung giả)
10	Sai sự thật nhưng đúng format (Plumbing)	`PASS (Plumbing)`	`Judge từ chối` (Mock correctness=0)

Bảng 3 (tiếp theo): *Kết quả kiểm thử 18 kịch bản*

STT	Kịch bản kiểm thử (Test Scenario)	Trạng thái	Điều kiện dừng / Kết quả
11	Thiếu nội dung bắt buộc (Plumbing)	`PASS (Plumbing)`	`Judge từ chối` (Mock completeness=0)
12	Bịa đặt nguồn, không bám sát (Plumbing)	`PASS (Plumbing)`	`Judge từ chối` (Mock groundedness=0)
13	Thay đổi `action_queue` khi REPLAN	`PASS (Replan)`	Kế hoạch mới được lưu log audit
Nhóm 2: Real LLM Integration Tests (5 kịch bản – Chạy với Ollama thật: 5/5 PASS)			
14	Viết bài RAG hoàn chỉnh với LLM thật	`PASS (Ollama)`	Hoàn thành 3 bước, Judge nghiệm thu
15	Adversarial (Real LLM): Lỗi Correctness	`PASS (Ollama)`	LLM Judge từ chối (Sai dữ kiện)
16	Adversarial (Real LLM): Lỗi Sai sự thật	`PASS (Ollama)`	LLM Judge từ chối (Sai hoàn toàn)
17	Adversarial (Real LLM): Lỗi Thiếu ý	`PASS (Ollama)`	LLM Judge từ chối (Thiếu nội dung)
18	Adversarial (Real LLM): Lỗi Bịa đặt nguồn	`PASS (Ollama)`	LLM Judge từ chối (Bịa đặt nguồn)

```
# [Lần chạy 1 - Có kết nối Ollama thực tế (tinix-lm:latest)]:
Ran 18 tests in 29.811s - OK (18/18 PASS: 13 Unit Tests PASS, 5 Real LLM Tests PASS)
# [Lần chạy 2 - Môi trường Offline độc lập (không có dịch vụ Ollama)]:
Ran 18 tests in 0.045s - OK (skipped=5, 13 Unit Tests PASS)
```

(Chú thích phân định: Khi có Ollama, 18 kịch bản (gồm 5 Integration Tests LLM Judge đối kháng) chạy hoàn tất trong 29.811s; khi không có Ollama, 5 tests tự động bỏ qua (`skipped=5`), 13 Unit Tests kiểm chứng hoàn tất trong 0.045s).

Phân tích báo cáo tổng kết và nhật ký từng bước thực thi

Sau khi kết thúc phiên làm việc, Agent tự động kết xuất tệp tin `agent_final_report.json`. Dưới đây là cấu trúc tệp tin kiểm toán thực tế được sinh ra (trích đoạn rút gọn minh họa các trường chính; xem tệp đầy đủ `agent_final_report.json` trên repository):

```
{
  "goal": "Viết một bài chia sẻ ngắn giải thích RAG là gì cho người mới bắt đầu.",
  "total_steps_planned": 3, "steps_executed": 3, "status": "COMPLETED",
  "stop_reason": "Goal achieved: Bài viết hoàn chỉnh đạt chuẩn định lượng và ngữ nghĩa của
↪  QualityEvaluator (LLM Judge Verified).",
  "plan": [{"step": 1, "title": "Nghiên cứu RAG"}, {"step": 2, "title": "Dàn ý bài viết"}, {"step": 3,
↪  "title": "Viết bài chia sẻ"}],
```

```
  "execution_logs": [{"step": 1, "duration_sec": 3.176}, {"step": 2, "duration_sec": 3.356}, {"step": 3,
↪   "duration_sec": 7.380}],
  "quality_metrics": {
    "passed": true, "quality_passed": true, "semantic_verified": true,
    "char_count": 1872, "word_count": 391, "ttr": 0.427, "matched_keywords": ["giải", "thích", "rag"],
    "judge_verdict": {
      "correctness": 1, "groundedness": 1, "completeness": 1,
      "reason": "Văn bản mô tả chính xác RAG kết hợp truy xuất và sinh nội dung, đúng bản chất.",
      "judge_model": "tinix-lm:latest", "groundedness_source": "none_open_domain", "judge_status":
↪   "VERIFIED_SUCCESS"
    }
  },
  "total_duration_sec": 19.812,
  "reproducibility": {"model_version": "tinix-lm:latest", "git_commit": "29cc9b1", "temperature": 0.25,
↪   "seed": null, "python_version": "3.8.10", "prompt_template_summary": "Lập checklist..."}
}
```


Báo cáo này mang lại giá trị cốt lõi về **Tính minh bạch (Observability)** khi thể hiện rõ thời gian thực thi (19.812s, làm tròn 19.8s), khóa ngữ cảnh tiêu thụ và lịch sử các bước. **Phân định nguồn Groundedness:** Giá trị `"none_open_domain"` cho bài viết RAG phản ánh trung thực đây là tác vụ giải thích tri thức mở (`input_document=None`), LLM Judge thẩm định theo tri thức chuẩn để chống bịa đặt, tuyệt đối không ngộ nhận đã đối chiếu tài liệu bên ngoài. Với tác vụ tóm tắt (`input_document` có sẵn), trường này chuyển thành `"external_document"` để đối chiếu trực tiếp văn bản nguồn; nếu thiếu nguồn, Agent chuyển sang `FAILED` (Test 9). Về **khả năng tái lập (reproducibility)**, hệ thống lưu vết đầy đủ siêu dữ liệu (`model_version`, `git_commit`, `temperature`, `seed`) hỗ trợ kiểm toán minh bạch.

Mở rộng và ứng dụng thực tế của tác nhân tự hành

Mô hình tác nhân hoàn thành checklist 3 bước có thể mở rộng linh hoạt cho nhiều bài toán thực tế:

1. **Trợ lý viết tài liệu API:** Khi có endpoint mới: Quét hàm bóc tách mã lỗi → Sinh tài liệu kèm ví dụ cURL → Rà soát chuẩn OpenAPI và lưu file.
2. **Trợ lý rà soát pull request:** Khi có PR mới: Trích xuất git diff → Đối chiếu quy tắc an ninh (secret, SQL injection) → Gửi comment phản hồi lên Git.
3. **Trợ lý tự động hóa CSKH:** Khi tiếp nhận khiếu nại: Tra cứu đơn hàng CSDL → Đối chiếu chính sách đổi trả → Soạn email phản hồi thấu cảm và cập nhật CRM.

Kết luận: Đánh giá phạm vi tự hành và hướng phát triển

Xây dựng hệ thống tác nhân tự hành (**Autonomous Systems**) không đòi hỏi những mô hình phức tạp, mà bắt đầu từ những nguyên lý kỹ thuật vững chắc: **Mục tiêu rõ ràng → Kế**

hoạch có cấu trúc → Thực thi tuần tự → Quan sát & Đánh giá → Retry/Replan → Điều kiện dừng chuẩn xác (trong giới hạn số bước an toàn).

Phạm vi tự hành và ranh giới nghiên cứu: Nhìn nhận một cách khách quan và trung thực, hệ thống Checklist Agent đã kiểm chứng thành công các năng lực cốt lõi: (1) *Lập kế hoạch động qua LLM*: tự động phân rã bước theo mục tiêu thay vì chuỗi cố định; (2) *Vòng lặp quan sát thích ứng*: thu nhận phản hồi để điều chỉnh chỉ dẫn bước kế tiếp và kích hoạt REPLAN khi gặp rào cản; (3) *Chốt chặn an toàn Bounded Loop*: khóa trần ≤ 3 bước và từ chối nghiệm thu (`BUDGET_EXHAUSTED`) khi chưa đạt chuẩn. Tuy nhiên, trong phạm vi bài học kỹ thuật này, hệ thống đóng vai trò mô hình nghiên cứu/thử nghiệm (sandbox prototype): các công cụ trong `ToolRegistry` hoạt động dưới dạng mô phỏng (mock), và cơ chế fallback khi LLM ngoại tuyến dựa trên luật xác định để bảo đảm an toàn. Đây là bước đệm kiến trúc thiết yếu trước khi tích hợp API thực tế và phát triển các hệ thống Multi-Agent phức tạp hơn.

Mã nguồn đầy đủ, Nhật ký chạy thật và Báo cáo kiểm toán (Repository): Toàn bộ mã nguồn, dữ liệu kiểm toán (`agent_final_report.json`) và nhật ký thực thi nguyên bản trên terminal (`test_execution_ollama.log`) được lưu trữ công khai tại GitHub: https://github.com/thaiptit123/autonomous-checklist-agent.

Tài liệu tham khảo

[1] **Anthropic**: *Building Effective Agents*. Ranh giới Workflow & Autonomous Agent. https://www.anthropic.com/research/building-effective-agents

[2] **LangChain / LangGraph Documentation**: Khung phát triển Agent có trạng thái. https://langchain-ai.github.io/langgraph/

[3] **Lewis et al.**: *Retrieval-Augmented Generation for NLP Tasks*. Nghiên cứu gốc RAG. https://arxiv.org/abs/2005.11401

[4] **ReAct Framework**: *Synergizing Reasoning and Acting in Language Models* (ICLR 2023). https://arxiv.org/abs/2210.03629

[5] **AI Guru x TiniX Tutorial Series**: Chuỗi tài liệu chuẩn cho kỹ sư AI.